

Seksjon 1

Om eksamen

DAT108 Programmering og Webapplikasjoner, høst 2024

Tid: 4 timer

Du kan svare på engelsk eller norsk. Skriver du på norsk kan du bruke engelske faguttrykk.

Jeg har lest og forstått instruksjonene

☐ Ja

☐ Nei

Oppgave 1 (10%) - Strømmer/Streams

Kort om retting:

- Teller ca. 10% (24 minutter).
- Fire deloppgaver: A, B, C og D.
- Hver deloppgave teller ca. 2,5% (6 minutter)
- Oppgaven rettes automatisk.
- Det er bare en **helt** rett løsning på hvert problem.
- Du blir gitt **delvis** uttelling for delvis korrekt svar:
 - Alle felt som er korrekte gir poeng.
 - Feil svar gir **IKKE** trekk.

Oppgave A:

Les hver beskrivelse nøye og velg det svaret som best passer til oppgaven.

i) Hvilken metode brukes til å fjerne duplikater fra en strøm?

- | | |
|---|-----------------------|
| a | <code>sorted</code> |
| b | <code>filter</code> |
| c | <code>distinct</code> |
| d | <code>limit</code> |
| e | <code>map</code> |

ii) Hva gjør `stream().filter(n -> n > 5)` ?

- | | |
|---|---|
| a | Fjerner de 5 største elementene i strømmen. |
| b | Beholder kun elementer større enn 5. |
| c | Multipliserer hvert element med 5. |
| d | Beholder de 5 første elementene i strømmen. |
| e | Endrer rekkefølgen på elementene. |

iii) Hvordan begrenser du antall elementer som behandles av strømmen til kun 3?

a `filter(n -> n < 3)`

b `limit(3)`

c `skip(3)`

d `reduce(3)`

e `map(x -> 3)`

iv) Hvis du vil kombinere alle strengene i en strøm til en streng. Hvilken metode bruker du?

a `map()`

b `reduce()`

c `sorted()`

d `filter()`

e `toList()`

v) Hvilken beskrivelse passer best for metoden `flatMap` i strømmer?

a Kombinerer alle elementene i strømmen til én enkelt verdi.

b Fjerner dupliserte elementer fra strømmen.

c Sorterer elementene i strømmen.

d Tar en strøm av samlinger og konverterer den til en enkelt strøm med alle elementene i samlingene.

e Filtrerer elementene i strømmen basert på en betingelse.

Oppgave B:

Gitt en liste med strenger (`List<String> ord`), gjør følgende:

1. Behold kun ord som starter med "A" eller "a".
2. Sorter ordene etter lengde (kortest først).
3. Fjern duplikater.
4. Legg til et utropstegn (`!`) på slutten av hvert ord.
5. Gjør om alle bokstaver i ordene til store bokstaver.

Til slutt samles resultatene i en ny liste (`List<String>`).

```
List<String> resultat = desimaltall.stream()
    .filter(
        .
    )
    .sorted(
        .
    )
    .distinct(
        .
    )
    .map(
        .
    )
    .map(
        .
    )
    .toList();
```

Oppgave C: Du skal bruke nedtrekksmenyene til å lage følgende strøm:

Julenissen trenger hjelp med å behandle en ønskeliste! Du har en **ønskeliste** (`List<Gave>`), som er en liste over gaver. Fullfør strømmen for å lage en liste over pakkene som Julenissen skal levere, basert på følgende kriterier:

1. Behold kun gaver som er på lager (`erPåLager` er `true`).
2. Sorter gavene etter prioritet (høyest først).
3. Pakk gavene inn som `Pakke` .
4. Tell antallet pakker med en vekt over 1 kg.

Du har følgende funksjonelle grensesnitt implementert fra før:

```
Predicate<Gave> erPåLager; // Sjekker om en gave er på lager.
```

```
Comparator<Gave> etterPrioritet; // Sorterer gavene etter prioritet, høyest først.
```

```
Function<Gave, Pakke> pakkInnSomPakke; // Gjør om en Gave til en Pakke.
```

```
Predicate<Pakke> erTungPakke; // Sjekker om en pakke veier mer enn 1 kilo.
```

```
long pakker = ønskeliste.stream()
    .filter(
        .
    )
    .sorted(
        .
    )
    .map(
        .
    )
    .map(
        .
    )
    .filter(
        .
    );
```

Oppgave D: Hvilket tall blir skrevet ut når koden under kjøres?

```
import java.util.Arrays;
import java.util.List;

public class StreamExample {
    public static void main(String[] args) {
        List<Integer> tall = Arrays.asList(5, 3, 8, 10, 9, 2, 4, 6, 7, 1);

        double resultat = tall.stream()
            .limit(7)
            .filter(n -> n % 2 != 0)
            .mapToInt(n -> n * n)
            .sorted()
            .average()
            .orElse(-1);

        System.out.println(resultat);
    }
}
```

Tallet som blir skrevet ut er

Antagelser/bemerkninger:

Dette feltet kan stå tomt. Dersom noe er uklart i oppgavene over så kan du skrive ned eventuelle antagelsene du har gjort her. Marker kommentarene dine med oppgavenummer. Skriv for eksempel:

// Oppgave B iii).

Dine antagelser...

B *I* U | ☰ ☷ á

0 / 10000 Word Limit

Oppgave 2 (10%) - Funksjonelle kontrakter

Kort om retting:

- Teller ca. 10% (24 minutter).
- Fire deloppgaver: A, B, C og D
- Hver deloppgave teller ca. 2.5% (6 minutter)
- Oppgaven rettes automatisk.
- Det er bare en **helt** rett løsning på hvert problem.
- Du blir gitt **delvis** uttelling for delvis korrekt svar:
 - Alle felt som er korrekte gir poeng.
 - Feil svar gir **IKKE** trekk.

Oppgave A: Gi rett type til variablene a1, a2, a3 og a4 under:

// i)

a1 = (s, t) -> s.length() > t.length();

// ii)

a2 = s -> s.substring(0, 3).toLowerCase();

// iii)

a3 = (s, t) -> s.length() - t.length();

// iv)

a4 = list -> list.stream().distinct().toList();

Oppgave B: Velg det lambdauttrykket passer med typen til variabelene b1, b2, b3 og b4.

```
// i)
Predicate<Oppskrift> b1 =  ;

// ii)
Function<Oppskrift, String> b2 =  ;

// iii)
Comparator<Oppskrift> b3 =  ;

// iv)
BiFunction<List<Oppskrift>, Integer, Integer> b4 =  ;
```

Informasjon om klassen Oppskrift:

Oppskrift har følgende feltvariabler og metoder:

Feltvariabler:

String navn	// Navnet på julekaken, f.eks., "Pepperkaker".
Boolean nøtter	// Sann dersom oppskriften inneholder nøtter
int tilberedningstidMin	// Total tilberedningstid i minutter.
int godhet	// En skala fra 1 til 10, hvor høyere tall betyr bedre smak.
String vanskelighetsgrad	// Vanskelighetsgradene er LETT, MIDDELS, og VANSKELIG.

Konstruktør:

En konstruktør som tar inn navn, sukkerGram, tilberedningstidMin, godhet, og vanskelighetsgrad.

Metoder:

```
boolean erEkstraGod(); // Returnerer true hvis godhet > 8, ellers false.
int sammenlignVanskelighet(Oppskrift annen); // Returnerer en negativ verdi hvis denne oppskriften er lettere, 0 hvis de er like, og en positiv verdi hvis den er vanskeligere.
```

Oppgave C: Velg rett lambdauttrykk og type til variablene c1, c2 og c3 som passer med beskrivelsen av input og output.

```
// i)
// Inn: En oppskrift (Oppskrift).
// Ut: En streng som inneholder oppskriftens navn og hvor lang tid den tar å lage.
```

c1 = ;

```
// ii)
// Inn: To oppskrifter (Oppskrift).
// Ut: Differansen i smak (godhet) mellom de to oppskriftene.
// Hint: Metoden skal senere brukes i en strøm til sortering av oppskrifter etter smak.
```

c2 = ;

```
// iii)
// Inn: En oppskrift (Oppskrift) og et tall som representerer gjenværende tid (i minutter).
// Ut: true dersom oppskriften tar mindre enn gjenværende tid og ikke inneholder nøtter.
```

c3 = ;

Oppgave D: Er kodelinjene under gyldig? (I.e. er syntaks korrekt OG er uttrykket gitt rett funksjonell kontrakt)?

		Gyldig	Ugyldig
a	<pre>Predicate<Oppskrift> lambda1 = oppskrift -> oppskrift.getTilberedningstidMin() < 30;</pre>	☐	☐
b	<pre>Supplier<Oppskrift> lambda2 = () -> new Oppskrift("Pepperkaker", true, 45, 8, "MIDDELS");</pre>	☐	☐
c	<pre>Function<Oppskrift, Oppskrift> lambda3 = (o1, o2) -> o1.getNavn().equals(o2.getNavn());</pre>	☐	☐
d	<pre>Consumer<Oppskrift> lambda4 = oppskrift -> System.out.println("Oppskrift: " + oppskrift.getNavn());</pre>	☐	☐
e	<pre>BiPredicate<Oppskrift, Integer> lambda5 = (oppskrift, terskel) -> oppskrift.getGodhet() > terskel;</pre>	☐	☐
f	<pre>Function<List<Oppskrift>, List<String>> lambda6 = oppskrifter -> oppskrifter.stream().filter(o -> o.getGodhet() > 5).map(Oppskrift::getNavn).toList();</pre>	☐	☐

Antagelser/bemerkninger:

Dette feltet kan stå tomt. Dersom noe er uklart i oppgavene over så kan du skrive ned eventuelle antagelsene du har gjort her. Marker kommentarene dine med oppgavenummer. Skriv for eksempel:
// Oppgave B iii).
Dine antagelser...

B*I*U | ☰ ☷ á

0 / 10000 Word Limit

Oppgave 3 (10%) - JavaScript teorispørsmål

I disse flervalgsoppgavene er ett og kun ett av utsagnene korrekt. En oppgave gir kun poeng hvis det korrekte utsagnet er valgt.

a) (1%)

En variabel *farge* skal tilordnes tekststrengen «hvit». Hvilket av alternativene under er riktig?

- ☐ `private String farge = "hvit";`
- ☐ `String farge = "hvit";`
- ☐ `let farge => () {return "hvit"};`
- ☐ `String farge = () {return "hvit"};`
- ☐ `let farge = "hvit";`
- ☐ `const farge = {"hvit"};`

b) (1%)

JavaScript koden under er brukt i en web-applikasjon:

```
let tid = "08:19";  
let [timer,minutter] = tid.split(":");
```

For hjelp om metoden *split*, se hjelp i eksamen til programmeringsoppgaven i JavaScript. Hvilket av utsagnene under er korrekt?

- ☐ Koden vil feile da konstruksjonen med «[]» ikke kan brukes på venstre side av et likhetstegn.
- ☐ Variabelen *tid* endres, der «08» erstattes med verdien fra variabelen *timer* og «19» erstattes med verdien fra variabelen *minutter*.
- ☐ Variablene *timer* og *minutter* blir begge av type **number**, der *timer* får verdi 8, og *minutter* får verdi 19.
- ☐ Variablene *timer* og *minutter* blir begge av type **string**, der *timer* får verdi «08», og *minutter* får verdi «19».
- ☐ Variabelen *timer* blir en **string** med verdi «08:19», og minutter blir *undefined*.
- ☐ Variabelen *timer* blir *undefined*, mens minutter blir et **number**, med verdi lik 499 (499=8·60+19).

c) (1%)

JavaScript og *private* felt, egenskaper og metoder

Hvilket av utsagnene under er korrekt?

- ☐ JavaScript har ikke *private* felt, egenskaper eller metoder. Et felt, en egenskap eller metode med et navn som starter med enten tegnet «_» eller «#» kan betraktes som *private*, men dette er kun konvensjon og har ingen praktisk betydning.
- ☐ I JavaScript opprettes *private* felt, egenskaper og metoder både for objekter og klasser ved å bruke nøkkelordet *private*.
- ☐ Kun konstruksjoner med nøkkelordet *class* kan ha *private* felt og metoder, og de opprettes ved å bruke nøkkelordet *private*.
- ☐ Et felt, en egenskap eller en metode med et navn som starter med tegnet «#» er *private* i JavaScript, både for klasser og objekter.
- ☐ Kun konstruksjoner med nøkkelordet *class* kan ha *private* felt og metoder. Felt og metoder som har et navn som starter med tegnet «#» er *private*.
- ☐ I konstruksjoner med nøkkelordet *class* er som standard alle felt og metoder *private*. Et *public* felt eller en *public* metode må ha et navn som starter med tegnet «#».

d) (1%)

Et objekt er opprettet ved følgende kode:

```
const medlem = {  
  "navn": "ole",  
  "dato": new Date(2019,10,12)  
}
```

Medlemmet sitt navn skal tilordnes en variabel *kasserer*. Hvilket av alternativene under er riktig?

- ☐ `let kasserer = medlem=>navn;`
- ☐ `let kasserer = medlem->navn;`
- ☐ `let kasserer = medlem.navn;`
- ☐ `let kasserer = medlem.getProperty("navn");`
- ☐ `var kasserer = medlem.getProperty("navn");`
- ☐ `let navn = medlem.getProperty("kasserer");`

e) (1%)

Følgende JavaScript kode er gitt:

```
const navn = "ole";  
const telefon = "111 22 333";  
const medlem = {navn, telefon};
```

Hva vil nå være innholdet i *medlem*?

- ☐ {"navn": "ole", "telefon": "111 22 333"}
- ☐ {"ole", "111 22 333"}
- ☐ {undefined: "ole", undefined: "111 22 333"}
- ☐ {null: "ole", null: "111 22 333"}
- ☐ {"ole":undefined, "111 22 333":undefined}
- ☐ {"ole":null, "111 22 333":null}

f) (1%)

Følgende JavaScript kode er gitt:

```
const uttrykk = /^.{3}$/;
```

Hvilket utsagn er riktig?

- ☐ Størrelsen *uttrykk* er et **RegExp** objekt som brukes for å finne mønstre i tekst.
- ☐ Størrelsen *uttrykk* er et **RegExp** objekt som brukes for å konvertere mellom JavaScript objekter og **String**.
- ☐ Størrelsen *uttrykk* er et **JSON** objekt som brukes for å konvertere mellom JavaScript objekter og **String**.
- ☐ Størrelsen *uttrykk* er en tekstmal, der «{ }» konstruksjoner erstattes ved å kjøre JavaScript kode.
- ☐ Størrelsen *uttrykk* er et **Intl** objekt som brukes for å internasjonalisere tekst.
- ☐ Koden vil feile da tekststrenger må omslutes av hermetegn.

g) (1%)

Et web dokument inneholder følgende HTML:

```
<input type="text" placeholder="Navn" id="navn">
```

Bruker fyller inn et navn i input-elementet. Hvordan kan denne verdien finnes med JavaScript kode?

- ☐ `const navn = document.getElementById("navn").innerHTML;`
- ☐ `const navn = document.querySelector("navn").innerHTML;`
- ☐ `const navn = document.getElementById("navn").value;`
- ☐ `const navn = document.querySelector("navn").value;`
- ☐ `const navn = document.getElementById("navn").textContent;`
- ☐ `const navn = document.querySelector("navn").textContent;`

h) (1%)

Et web dokument inneholder følgende HTML:

```
<span class="fornavn">01e<span>
```

Med *spanElm* et objekt som representerer HTML `<span>` elementet, hvilket av alternativene under lar oss hente ut verdien *fornavn* fra *class* attributtet?

- ☐ `spanElm.class`
- ☐ `spanElm.classList`
- ☐ `spanElm.classList[0]`
- ☐ `spanElm.getAttribute("classList")`
- ☐ `spanElm.className[0]`
- ☐ `spanElm.getAttribute("className")`

i) (1%)

Et HTML `<table>` element har diverse egenskaper i JavaScript. Velg alternativet under med den største gruppen av egenskaper som finnes for et HTML `<table>` element.

- ☐ `tHead, tBody`
- ☐ `tHead, tBodies`
- ☐ `tHead, tBody, rows`
- ☐ `tHead, tBodies, rows`
- ☐ `tHead, tBody, rows, cells`
- ☐ `tHead, tBodies, rows, cells`

j) (1%)

Følgende JavaScript kode er gitt:

```
function beregn(...args) {  
  if (args.length === 0) {  
    return 0;  
  } else {  
    const [arg0,...rest] = args;  
    return arg0 + beregn(...rest);  
  }  
}
```

Hvilket utsagn er riktig?

- ☐ Koden vil feile da «...» er brukt feil.

Funksjonen *beregn* må gis en tabell av tall som parameter, som tilordnes *args* og som summeres:

- ☐ `const resultat = beregn([1,2,3]);`
`// resultat er 6`

Funksjonen *beregn* må gis tall som parametre, som legges i tabellen *args* og som summeres:

- ☐ `const resultat = beregn(1,2,3);`
`// resultat er 6`

Funksjonen *beregn* må gis en **string** med tall som parameter, som legges i en **string** *args* og som summeres som tall og returneres som en **string**:

- ☐ `const resultat = beregn("1,2,3");`
`// resultat er "6"`

Funksjonen *beregn* må gis en **string** som parameter, og som returnerer strengen reversert:

- ☐ `const resultat = beregn("ABC");`
`// resultat er "CBA"`

Funksjonen *beregn* returnerer parametrene gitt til funksjonen:

- ☐ `const resultat = beregn("A",1,"B");`
`// resultat er "A",1,"B"`

Antagelser/bemerkninger:

Dette feltet kan stå tomt. Dersom noe er uklart i oppgavene over så kan du skrive ned eventuelle antagelsene du har gjort her. Marker kommentarene dine med oppgavenummer. Skriv for eksempel:

// Oppgave b).

Dine antagelser...

B	<i>I</i>	<u>U</u>		☰	¹ / ₃ ☰	á
						0 / 10000 Word Limit

Se oppgave på papir. For å svare, velger du «Administrer vedlegg». Så

Da får du åpnet en ny fane der du skriver koden.

1. Start med å gi vedlegget et navn. For eksempel "Oppgave 4"
2. Klikk lagre. Da aktiverer du automatisk lagring.
3. Start å skrive kode

B

I

U

|

☰

☷

á

0 / 10000 Word Limit

Oppgave 5 (15%) - JavaScript program

Se oppgave på papir. For å svare, velger du «Administrer vedlegg». Så

Nytt vedlegg
Kode
Kodespråk - JavaScript

Da får du åpnet en ny fane der du skriver koden.

For å sikre automatisk lagring, følg denne oppskriften

- 1. Start med å gi vedlegget et navn. For eksempel "Oppgave 5"
- 2. Klikk lagre. Da aktiverer du automatisk lagring.
- 3. Start å skrive kode

Når du har svart, går du tilbake til hovedfanen.

B

I

U

☰

☷

á

10000 Word Limit

Kode (som du kan klippe og lime inn i besvarelsen din)

I oppgaven skal dere fylle inn den manglete koden i en JavaScript-klasse **Trafikkontroller**. Se oppgavetekst på papir for detaljer.

```
class Trafikkontroller {
  #nedregrense;
  #ovregrense;
  #tbodyelement;
  #billiste = [];

  constructor(felt1, felt2) {
    // Legg inn kode for felt 1, felt for å registrere bilpassering

    // Legg inn kode for å vise dagens dato i felt1 sitt <legend>

    // Kode for felt2, felt for å liste bilpasseringer
    this.#nedregrense = felt2.getElementsByTagName("input")[0];
    this.#ovregrense = felt2.getElementsByTagName("input")[1];
    this.#tbodyelement = felt2.getElementsByTagName("tbody")[0];
    felt2.querySelector("button").addEventListener(
      "click",
      () => { this.#vispasseringer() }
    );
  }

  #vispasseringer() {
    // Legg inn kode her for å liste bilpasseringer
  }

  // Legg til metode for å registrere en bilpassering

  // Legg til eventuelle hjelpemetoder
}

const felt1 = document.getElementById("registrering");
const felt2 = document.getElementById("resultat");

new Trafikkontroller(felt1,felt2);
```

Oppgave 6 (10%) - Passord

Se oppgave på papir. Du kan skrive svaret ditt direkte i boksen under.

</>

Oppgave 7 (30%) - Web backend med Spring MVC

Se oppgave på papir. For å svare, velger du «Administrer vedlegg». Så

Nytt vedlegg

Kode

Kodespråk - Java

Da får du åpnet en ny fane der du skriver koden.

For å sikre automatisk lagring, følg denne oppskriften

1. Start med å gi vedlegget et navn. For eksempel "Oppgave 6"
2. Klikk lagre. Da aktiverer du automatisk lagring.
3. Start å skrive kode

Når du har svart, går du tilbake til hovedfanen.

</>